INTERACTIVE SINHALA TEXT READER FOR THE VISUALLY IMPAIRED USING OCR, NLP, AI

INDIVIDUAL THESIS

Muthunayakage Janindu Jithnuka Muthunayaka

(IT22072238)

Topological Sinhala Character Recognition via Medial Axis Transform

Group ID: R26-DS-002

Report submitted in partial fulfilment of the requirements for the

B.Sc. in Information Technology

Specializing in Data Science

Department of Computer Science

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Muthunayakage Janindu Jithnuka Muthunayaka

Student ID: IT22072238

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Lakmini Abeywardhana		
Co-supervisor: Adya Dissanayaka		

ABSTRACT

This research evaluates Medial Axis Transform (MAT) based skeletonization as a preprocessing step for Sinhala optical character recognition (OCR), developed as the layout analysis and headline recognition component for the Group R26-DS-002 assistive text reader. While thinning and skeletonization were central to OCR in the pre-deep-learning era, most contemporary Sinhala OCR systems rely on end-to-end feature learning without revisiting whether topological preprocessing offers benefits for stylized Sinhala typography. This study investigates that gap to serve visually impaired and elderly readers excluded from printed information by standard OCR engines.

A controlled design evaluated a standard recognition pipeline (Referred to as RAW) against a topologically preprocessed pipeline (referred to as MAT), both built on custom-trained Tesseract LSTM models. The evaluation spanned four complementary types of evidence: a large-scale benchmark on 8,872 images held-out from the SinOCR dataset, a stress test across 135 structurally difficult fonts (generated into 10,000 images), an objective geometric edge-density analysis, and a human perceptual survey consisting of 404 trials. Results establish that the RAW pipeline is the stronger and faster generalist overall (mean CER 0.0586 vs. 0.0922, $p < 0.001$; ~213 ms vs. 5,777 ms per crop). Conversely, the MAT pipeline demonstrated lower CER on a narrow, geometrically distinct subset of 29 fonts that were measured to be more complex by edge-density analysis (1.59 vs 1.25; $p = 0.0067$).

Since the significant processing time increase rules out MAT as a universal replacement, these findings motivate an hybrid, confidence-gated routing architecture. The final product pipeline combines PaddleOCR layout segmentation, CRAFT text-line detection and the hybrid router, where RAW serves as the default and MAT is invoked as a fallback for high-complexity cases. This individual contribution delivers the complete layout segmentation and headline recognition pipeline alongside the custom models, concluding MAT as a specialist branch for structurally complex Sinhala typography.

Keywords: Sinhala OCR, Medial Axis Transform, Tesseract LSTM, document layout analysis, assistive technology

ACKNOWLEDGEMENTS

I express my sincere gratitude to my supervisor, Lakmini Abeywardhana, and co-supervisor, Adya Dissanayaka, for their guidance, advice and encouragement throughout this research and development work. I also would like to thank the members of Group R26-DS-002, Bumal Sathsara, Ishara Madusanka and Nadee Devindi for their collaboration in integrating the components of the assistive Sinhala text reader as well as the support provided throughout this project. Finally I would like to kindly acknowledge the 21 volunteers who took part in the perceptual survey, and the academic and technical staff of the Faculty of Computing for their support.

Table of Contents

1 INTRODUCTION 1

1.1 Overall Project Context 1

1.2 Individual Component Context 1

1.3 Problem Statement 2

1.4 Research Gap 2

1.5 Aim and Objectives 3

1.6 Research Questions or Hypotheses 3

1.7 Scope Limitations and Assumptions 4

1.8 Individual Contribution 4

1.9 Thesis Structure 5

2 LITERATURE REVIEW AND RELATED WORK 6

2.1 Review Method and Scope 6

2.2 Domain and Theoretical Foundations 6

2.3 Related Research 6

2.4 Existing Systems Products and Techniques 7

2.5 Gap Synthesis and Conceptual Framework 7

3 RESEARCH METHODOLOGY 8

3.1 Research Design 8

3.2 Data Participants or Study Materials 8

3.3 Experimental or Study Procedure 9

3.4 Measures and Evaluation Metrics 10

3.5 Data Analysis 10

3.6 Ethics Privacy Security and Responsible AI 10

3.7 Reproducibility and Research Artefacts 11

4 PRODUCT AND SOLUTION DESIGN 12

4.1 Requirements 12
4.2 Component Architecture Within the Complete Solution 13
4.3 Design Alternatives and Decisions 14
4.4 Detailed Design 16
4.5 Standards and Professional Practice 18
5 IMPLEMENTATION INTEGRATION AND TESTING 20
5.1 Development Environment and Technology Stack 20
5.2 Component Implementation 21
5.3 Integration with the Other Components 23
5.4 Verification and Software Testing 25
5.5 Deployment and Operation 26
6 RESULTS AND VALIDATION 27
6.1 Research Results 27
6.2 Product and Technical Results 28
6.3 User or Stakeholder Validation 30
6.4 Integration Results 31
6.5 Summary of Findings 31
7 DISCUSSION 32
7.1 Interpretation of Findings 32
7.2 Comparison with Literature Baselines and Existing Solutions 32
7.3 Research Contribution 32
7.4 Product Contribution 34
7.5 Limitations and Threats to Validity 35
7.6 Implications 37
8 COMMERCIALISATION AND SUSTAINABILITY 38
8.1 Value Proposition and Target Users 38

8.2 Market and Competitive Position 38
8.3 Business or Sustainability Model 38
8.4 Scalability Intellectual Property and Compliance 39
8.5 Contribution to the Sustainable Development Goals 39
9 CONCLUSION AND FUTURE DIRECTIONS 40
9.1 Conclusion 40
9.2 Achievement of Objectives 40
9.3 Individual Research and Product Contributions 40
9.4 Future Directions 41
REFERENCES 43
GLOSSARY 45
APPENDICES 46
Appendix A: Research Instruments and Extended Results 46
Appendix B: Academic Integrity and AI Use Disclosure 47

LIST OF FIGURES

Figure 1: Processing flow of Component 1 within the complete solution 14
Figure 2:Integration of Component 1 with the other components of Group..... 25

LIST OF TABLES

Table 1: Training configuration of the sin_raw and sin_custom models 11
Table 2:Baseline evaluation on the common 8,872-image SinOCR subset 27
Table 3:Adversarial stress test results (135 fonts, 10,000 images) 27
Table 4:Geometric edge-density complexity of the font groups 27
Table 5:Processing latency on the AMD Ryzen 5 7235HS CPU 28
Table 6: Mapping of Component 1 layers to source modules 46
Table 7: Disclosure of AI and automated tool use 47

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
BCER	Best (Tesseract) Character Error Rate during training
BPE	Byte-Pair Encoding
CER	Character Error Rate
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CRAFT	Character Region Awareness for Text Detection
CTC	Connectionist Temporal Classification
GPU	Graphics Processing Unit
HTTP	Hypertext Transfer Protocol
IEEE	Institute of Electrical and Electronics Engineers
LAN	Local Area Network
LLM	Large Language Model
LSTM	Long Short-Term Memory
MAT	Medial Axis Transform
NGO	Non-Governmental Organization
NLP	Natural Language Processing
OCR	Optical Character Recognition
OEM	OCR Engine Mode
PSM	Page Segmentation Mode
RAG	Retrieval-Augmented Generation
SDG	Sustainable Development Goal
TSV	Tab-Separated Values
TTS	Text-to-Speech
TrOCR	Transformer-based Optical Character Recognition
UN	United Nations
VRAM	Video Random Access Memory
WHO	World Health Organization
ZWJ	Zero Width Joiner
mT5	Multilingual Text-to-Text Transfer Transformer

LIST OF APPENDICES

Appendix	Description	Page
Appendix A	Research Instruments and Extended Results	42
Appendix B	Academic Integrity and AI Use Disclosure	43

1 INTRODUCTION

1.1 Overall Project Context

The research is situated within Group R26-DS-002, which is tasked with developing an interactive Sinhala text reader to provide daily informational access for visually impaired and elderly demographics. This demographic is heavily reliant on printed materials for daily information. By extracting complex Sinhala typography with high accuracy, the system grants these individuals autonomous access to newspapers, textbooks, and formal documents. The broader system architecture integrates document layout segmentation, topological optical character recognition (OCR), context-aware error correction utilizing an mT5 model, retrieval-augmented generation (RAG) through a ChromaDB vector database, and a personalized voice interaction module into a deployable assistive framework. The deployment of this assistive Sinhala reader aligns directly with the United Nations Sustainable Development Goals (SDGs), specifically SDG 10 for reduced inequalities and SDG 4 for quality education by providing blind students with the autonomous ability to read printed Sinhala educational materials.

1.2 Individual Component Context

This specific component (Component 1) constitutes the visual ingestion and primary text extraction engine of the system, governing the pipeline across Layer 3 and Layer 4A. It executes Document Layout Analysis, Preprocessing, and Sinhala Headline Text Recognition. The component successfully navigates the complexities of Sinhala document layouts by accepting unconstrained document camera frames and segmenting them into distinct structural article blocks. Specifically, raw camera frames are first processed using PaddleOCR for robust macro-level text detection and CRAFT TextDetection to generate precise micro-level character and word affinity heatmaps. This isolates structural regions and prevents the mechanical clipping of multi-positional Sinhala vowel diacritics. Following this extraction, the primary focus shifts to recognizing highly stylized Sinhala headline typography using Medial Axis Transform (MAT) skeletonization paired with a custom Tesseract LSTM model. The

component operates exclusively on headline crops, concurrently running a standard binarized (RAW) branch and a MAT branch in parallel to select the most confident output. This ensures that the broader assistive reader ecosystem can ingest complete newspaper pages without catastrophic failures on stylized headline typography.

1.3 Problem Statement

Visually impaired Sinhala speakers remain systematically excluded from printed information due to the absence of affordable, script-aware assistive technology tailored to the local demographic. Currently available commercial assistive technologies, such as JAWS and Google Lookout, are predominantly English-centric and fail to provide functional, robust support for the complex Sinhala script. Compounding this technological void is the fundamental inadequacy of conventional Optical Character Recognition (OCR) engines when applied to structural complexities. Regular character detection engines frequently fail on complex Sinhala typography, particularly when parsing touching conjunct consonants (bandi akuru) and multi-positional vowel modifiers (pili) that are rendered in non-standard or highly stylized headline fonts. In printed media, Sinhala headlines are set in large, decorative, or stylized typefaces, and their vowel diacritics can be placed to the left, right, above, or below the base consonant. When conventional systems attempt to parse these dense, high-variance geometries, they typically use simple bounding boxes that mechanically clip these essential marks. The OCR then returns corrupted or incomplete words, generating errors that carry through every subsequent processing stage. The absence of proper recognition architectures capable of generalizing across these intricate typographic features leaves a critical gap in the assistive landscape, effectively barring visually impaired individuals from independent access to everyday print media.

1.4 Research Gap

The broader OCR field largely abandoned topological preprocessing methodologies when transitioning to neural feature learning, and the rapid evolution into end-to-end deep learning feature extraction bypassed the intermediate skeletonization era almost entirely. Because large-scale Sinhala OCR research only began comprehensively during the modern deep-learning era, existing baseline models heavily favor end-to-

end architectures and feed raw binarized pixel images directly into standard learned OCR engines. Consequently, no prior Sinhala-specific OCR research evaluates whether reintroducing established topological preprocessing methodologies, such as Medial Axis Transform (MAT), provides measurable accuracy benefits when placed ahead of a modern LSTM OCR architecture. The unique structural complexity of Sinhala suggests that explicit topological representations could yield significant benefits, yet contemporary models assume that modern neural engines automatically capture all necessary structural signals from raw pixels without explicitly testing skeletal preprocessing.

1.5 Aim and Objectives

The primary aim of this research is to empirically evaluate Medial Axis Transform (MAT) skeletonization as a topological preprocessing stage for Sinhala optical character recognition ahead of a modern LSTM engine. This objective is achieved by deploying a controlled, single-variable experimental design that isolates the impact of topological preprocessing. The study benchmarks a standard baseline recognition pipeline (RAW), which uses unmodified binarized images, against a topologically preprocessed pipeline (MAT). Both pipelines utilize a shared Tesseract LSTM architecture to ensure comparability. The comprehensive evaluation spans four independent lines of evidence: a large-scale baseline evaluation on an 8,872-image held-out subset of the SinOCR dataset, a human perceptual survey consisting of 404 trials, a 10,000-image synthetic adversarial stress test across 135 complex fonts, and an objective algorithmic geometric complexity analysis.

1.6 Research Questions or Hypotheses

The central hypothesis driving this research posits that Medial Axis Transform (MAT) skeletonization normalizes severe font variation and stroke width disparities, producing a standardized, universal structural representation. By stripping away extraneous stylistic noise and reducing characters to their core geometric topology, this preprocessing stage is hypothesized to significantly improve character recognition accuracy specifically for highly complex, non-standard Sinhala headline typography. Because Sinhala heavily relies on multi-positional modifiers and intricate conjuncts,

the assumption is that an explicit skeletal representation will capture the essential topological signals that purely learned pixel-level architectures might fail to generalize across during inference.

1.7 Scope Limitations and Assumptions

The primary operational boundary of this component is strictly centered on the visual ingestion, identification, extraction, and recognition of highly stylized Sinhala headline text. The scope encompasses the necessary preparatory visual pipeline from the initial acquisition of raw image frames through document layout segmentation to successfully isolate the target headline regions. The component's responsibility concludes upon the synthesis of the final headline string. All subsequent downstream processes are explicitly externalized to other modules within the ecosystem; this includes text assembly, context-aware error correction via mT5, text summarization, and text-to-speech synthesis, which are entirely managed by Components 2, 3, and 4. Methodologically, the evaluation is constrained by its reliance on a 135-font synthetic adversarial stress-test dataset rather than utilizing real-world, physically degraded, or noisy document samples. Furthermore, current system validation operates under the assumption of developer-simulated visual impairment acting as a proxy for usability and accessibility testing. Formal in-situ trials involving actual blind or low-vision users have been explicitly deferred to future iterations of the system's deployment lifecycle.

1.8 Individual Contribution

The core individual technical contribution encompasses the complete engineering and implementation of the two-layer visual ingestion and recognition pipeline, specifically Layer 3 (l3_segment) and Layer 4A (l4a_title). Within the layout analysis phase, this contribution includes the integration of macro-level PaddleOCR layout splitting paired with dynamic safety buffering, as well as micro-level CRAFT text-line focusing to accurately isolate headline bounding boxes without clipping diacritics. For the text extraction phase, the contribution centers on developing the four-stage headline recognition sequence in l4a_title, which includes the Medial Axis Transform (MAT) skeletonization and dilation sequence. Crucially, this contribution also covers the data preparation and complete training lifecycle of the custom Tesseract LSTM sequence-

to-sequence models (sin_custom and sin_raw) required to execute the dual-branch topological text recognition.

1.9 Thesis Structure

The thesis follows a structured progression to detail the research and product development lifecycle. Chapter 2 synthesizes theoretical foundations and the Sinhala OCR literature. Chapter 3 outlines the quantitative methodology and experimental datasets. Chapter 4 details the product architecture, rejected alternatives, and hybrid routing. Chapter 5 documents system implementation and component integration. Chapter 6 presents the comparative Character Error Rate (CER) results and validation metrics. Chapters 7, 8, and 9 cover the discussion, commercialization strategy, and concluding future directions.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

The literature review systematically examines optical character recognition (OCR) applied to low-resource South Asian scripts, with a specific focus on Sinhala [4]. The scope encompasses historical topological thinning methodologies, the evolution of Medial Axis Transform (MAT) in document image analysis, and contemporary end-to-end neural network architecture, specifically LSTM-based sequence-to-sequence models utilized in modern text recognition pipelines.

2.2 Domain and Theoretical Foundations

OCR research historically progressed through three distinct representational eras: systems built on raw pixel intensity, systems utilizing thinning and skeletonization to extract structural stroke features, and contemporary systems built on fully learned end-to-end neural feature extraction. Foundational thinning methodologies aim to extract one-pixel-wide skeletal representations that preserve the essential topology of character strokes [9]. MAT-based thinning strategies were subsequently developed to address artifacts like spurious branches that plagued naive thinning algorithms; however, foundational algorithmic evaluations of MAT primarily targeted English and Bengali character images, not Sinhala [10], [11]. The theoretical value of topology-aware representations persists in contemporary computer vision, where stroke topology is increasingly treated as a first-class structural signal rather than merely a pixel-level feature [12], indicating that skeletal structure carries information that purely learned pixel-level architectures may not automatically capture.

2.3 Related Research

Because large-scale Sinhala OCR research only began comprehensively during the modern deep-learning era, the field largely bypassed the structurally grounded skeletonization era that benefited higher-resource languages. Recent related research has heavily favored end-to-end architectures, including deep learning-based Sinhala OCR models [5], comprehensive benchmarking studies for Sinhala and Tamil document digitization [6], and the development of mobile Sinhala book readers for

visually impaired students built upon modern OCR and text-to-speech engines [7]. Crucially, the establishment of the SinOCR and SinFUND benchmarks [8], [15] has provided the standardized datasets necessary for evaluating modern models., demonstrating baseline performances for architectures like Tesseract and TrOCR on printed Sinhala text.

2.4 Existing Systems Products and Techniques

Commercially available assistive reading technologies, such as JAWS and Google Lookout, remain English-centric and offer no functional, robust support for the Sinhala script. While academic prototypes for assistive readers exist [7], [14], they generally ingest raw binarized pixel images directly into standard learned OCR engines. These existing techniques do not employ structural or topology-aware preprocessing, limiting their efficacy when confronted with the highly stylized, structurally complex conjuncts often found in Sinhala headline typography.

2.5 Gap Synthesis and Conceptual Framework

A synthesis of the literature reveals a significant, unaddressed gap: to the best of current knowledge, no prior Sinhala-specific OCR research evaluates the impact of skeletonization-based preprocessing ahead of a modern trainable OCR engine. The structural complexity of Sinhala of touching conjunct consonants and multi-positional vowel signs (*pili*) suggests that topological preprocessing could yield significant benefits. The conceptual framework of this research directly targets this gap by empirically evaluating whether an established topological representation (MAT) improves or degrades the recognition accuracy of a contemporary LSTM-based recognizer specifically for the Sinhala script.

3 RESEARCH METHODOLOGY

3.1 Research Design

The research employs a highly controlled, single-variable experimental design tailored to specifically isolate the impact of topological preprocessing on OCR accuracy. The core experimental setup consists of two parallel optical character recognition (OCR) pipelines: a standard baseline pipeline (RAW) and a topologically preprocessed pipeline (MAT), which differ fundamentally in the preprocessing stage under test. To ensure comparability, both pipelines are evaluated using separately trained variations of the same underlying Tesseract LSTM model engine. The design acknowledges structural differences in the benchmark evaluation, specifically the use of different page segmentation modes (PSM 13 for RAW versus PSM 8 for MAT). To validate the central hypothesis, the experimental pipelines are rigorously evaluated across four independent lines of evidence: a large-scale algorithmic benchmark conducted on a held-out evaluation split, a comprehensive human perceptual visual difficulty survey, an adversarial synthetic font stress test, and an objective algorithmic geometric complexity analysis.

3.2 Data Participants or Study Materials

The primary training and baseline evaluation material is the SinOCR-Printed benchmark dataset, which contains 100,000 printed Sinhala images rendered in 200 fonts [8], [15]. The dataset's official partition is a fixed 90,000-image training set and a 10,000-image held-out test set. The final Tesseract LSTM models (`sin_raw` and `sin_custom`) were trained on the full 90,000-image training set. The baseline evaluation uses the 8,872 images of the held-out test set that survived preprocessing in both pipelines (the common evaluation subset): all test images were resized to a height of 384 pixels with the aspect ratio preserved, and a number of images were lost independently in the RAW and MAT paths because of resizing anomalies and skeletonization edge cases (1,128 of the 10,000 test images were lost this way across the two paths). Because the two pipelines failed on different images, the evaluation was restricted to the intersection of the surviving sets so that the RAW-versus-MAT comparison is strictly one-to-one. An earlier, abandoned TrOCR trial (Section 4.3)

used a separate 10,000-image subset (9,000 training / 1,000 testing); it is not part of the reported results. To test robustness on structurally complex typography, a secondary synthetic stress-test dataset of 10,000 images was generated using 135 Sinhala fonts that the author collected from various sources and selected because they looked visually complex. This selection was made independently of how the 200 SinOCR fonts were chosen, so some overlap between the two font sets cannot be ruled out. The SinOCR test images and the stress-test images are pre-segmented word or short-phrase crops. For the human visual complexity survey, 21 volunteers from the general public (sighted, not visually impaired) judged 404 unique paired image trials (overall 808 image).

3.3 Experimental or Study Procedure

In both pipelines, images are resized to a target height of 384 pixels, maintaining the original aspect ratio, and binarized using Otsu thresholding [19].

- **RAW Pipeline:** The binarized crops are passed directly into a custom Tesseract LSTM model (sin_raw) trained exclusively on unmodified binarized Sinhala images, utilizing page segmentation mode (PSM) 13 (raw single line). In the benchmark evaluation the input is a pre-segmented word or short-phrase crop; in production it is the full stitched headline strip, which PSM 13 lets Tesseract treat as one line without multi-line layout analysis.
- **MAT Pipeline:** The binarized crops are additionally processed using the Zhang-Suen thinning algorithm [18] to extract a one-pixel-wide medial axis skeleton, which is then dilated by one pixel on each side to create a standardized 3-pixel stroke width. These skeletonized images are passed into a separate Tesseract model (sin_custom) trained on MAT-processed data, utilizing PSM 8 (single word) in the benchmark evaluation. Stated design rationale for PSM 8 (not independently tested): because the benchmark crops are already isolated words, Tesseract does not need to find word boundaries, and PSM 8 is intended to stop skeletonization-widened gaps inside complex conjuncts from being misread as spaces that fracture a word. In production, MAT receives the stitched multi-word headline strip, so both sin_raw and sin_custom are run with

PSM 13; the PSM difference therefore applies only to the benchmark evaluation.

3.4 Measures and Evaluation Metrics

The primary quantitative metric for recognition quality is the Character Error Rate (CER), computed via Levenshtein edit-distance between the ground-truth text and the model prediction. System feasibility is measured via end-to-end computational processing latency benchmarks (measured in milliseconds per crop). Geometric font complexity is objectively measured by computing the ratio of edge pixels to total foreground pixels using Canny edge detection. The subjective metric is human-perceived visual difficulty, measured via a 404-trial perceptual survey.

3.5 Data Analysis

Statistical significance for the baseline evaluation is determined using a paired, two-tailed t-test on per-image CER. The 135-font adversarial stress test is analyzed using a paired, two-tailed t-test on per-font average CER. The difference in edge-density complexity between the MAT-favored and RAW-favored font groups is tested with Welch's unequal-variance two-sample t-test. The results of the human perceptual survey are evaluated using an exact two-tailed binomial test against a 50% null hypothesis to determine whether human judgments of difficulty align with MAT pipeline victories.

3.6 Ethics Privacy Security and Responsible AI

The capture architecture enforces real-time transparency by broadcasting explicit audio cues through the Android TextToSpeech API to notify the user whenever the camera acquires a frame. To protect user data and sensitive document content, the processing pipeline utilizes an ephemeral, zero-retention memory architecture where raw images and text crops are discarded immediately post-synthesis. Furthermore, the system architecture supports local edge-hosting on a personal PC, guaranteeing that private documents never leave the user's local network. For the perceptual survey, participants gave informed consent for their email addresses to be collected securely,

solely to ensure that each trial was unique and to prevent duplicate voting; no personally identifiable information was linked to the published results.

3.7 Reproducibility and Research Artefacts

To support reproducibility, the 90,000 / 10,000 train/test partition used is the official SinOCR-Printed partition, and sin_raw and sin_custom were trained on the same 90,000 source images (sin_custom on their MAT-processed equivalents) and evaluated on the same test images. Both models were fine-tuned with tesstrain (make training) from the pre-trained Sinhala model (START_MODEL=sin) with default tesstrain learning-rate settings and no explicit overrides; the two runs did not end at the same iteration count: sin_raw plateaued at 82,110 iterations (best evaluation BCER 7.505% at iteration 79,965; final training BCER 7.955%), whereas sin_custom ran to 1,056,361 iterations (best evaluation BCER 10.558% at iteration 1,044,490; final training BCER 9.888%). Training in WSL was interrupted by environment crashes and resumed from checkpoints. These BCER values are Tesseract's internal character error rates during training, not the held-out CER reported in Chapter 6.

Table 3.1 summarizes the configuration and outcome of the two Tesseract training runs.

Table 1: Training configuration of the sin_raw and sin_custom models

Item	sin_raw (RAW)	sin_custom (MAT)
Training input	Binarized SinOCR images (90,000)	MAT-processed equivalents of the same 90,000 images
Start model	sin (pre-trained)	sin (pre-trained)
Training iterations	82,110	1,056,361
Best evaluation BCER	7.505% (iteration 79,965)	10.558% (iteration 1,044,490)
Final training BCER	7.955%	9.888%
PSM in benchmark	13 (raw single line)	8 (single word)
PSM in production	13 (raw single line)	13 (raw single line)

Source: Author’s training logs. BCER values are Tesseract’s internal training error rates, not the held-out CER of Chapter 6.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The design of Component 1 is governed by specific functional and non-functional requirements to ensure seamless document ingestion within the assistive text reader ecosystem:

- **Functional Requirements:**
  - *Document Layout Segmentation:* The component must accept full, unconstrained document camera frames and segment them into distinct structural article blocks.
  - *Headline and Body Disentanglement:* The pipeline must accurately differentiate headline/title regions from running body text paragraphs across varied column layouts.
  - *Sub-Region Boundary Preservation:* Bounding box generation must enforce safety buffer margins to prevent the mechanical clipping of ascenders, descenders, and multi-positional Sinhala vowel diacritics (*pili*).
  - *Sequence Flattening and Line Stitching:* Multi-line or curved headline regions must be normalized, perspective-corrected, and concatenated into standardized horizontal image strips.
  - *Topological Preprocessing and Recognition:* The system must binarize, optionally skeletonize, and transcribe stylized Sinhala headline text into Unicode strings using custom optical character recognition (OCR) models.
- **Non-Functional Requirements:**
  - *Inference Latency:* The pipeline must minimize computational overhead to preserve an interactive experience for visually impaired users.
  - *Architectural Modularity:* Visual processing layers must communicate via standardized schemas (`Article` and `Region` objects) to decouple visual extraction from downstream natural language processing (NLP) modules.

- *Privacy and Data Ephemerality:* Captured image buffers and intermediate crops must be processed entirely in volatile memory and purged immediately post-recognition to prevent local or remote storage of sensitive document data.

4.2 Component Architecture Within the Complete Solution

Within the group architecture (Group R26-DS-002), Component 1 governs the visual ingestion pipeline, spanning **Layer 3 (l3_segment)** and **Layer 4A (l4a_title)**:

- *Upstream Input:* Receives the selected high-quality frame from Layer 2 (l2_select).
- *Layer 3 (l3_segment) Routing:* Executes macro-level page layout analysis. Once article boundaries are defined, body text regions are extracted and routed downstream to Component 2 (l4b_body) for standard OCR and mT5 error correction. Concurrently, headline regions are routed into Layer 4A.
- *Layer 4A (l4a_title) Processing:* Operates exclusively on headline crops. It applies micro-level character focusing, binarization, and then runs a RAW branch and a MAT (Medial Axis Transform skeletonization) branch in parallel, each with its own custom Tesseract model; a confidence comparison selects the output.
- *Downstream Handoff:* Layer 4A populates the Article schema with title_raw (RAW output), title_mat (MAT output) and title_hybrid (the confidence-selected string). These are passed to Layer 4C (l4c_polish), a shared layer closely related to Component 3 (not part of Component 1's evaluated scope), which uses the document's body text as context to reconcile the RAW and MAT hypotheses with an LLM (dual-hypothesis fusion) into title_polished. Layer 5 (l5_assemble) applies the precedence title_polished > title_hybrid > title_raw, and reunites the headline with the processed body paragraphs to form structured articles for the RAG generator (Component 3) and conversational personalization layer (Component 4). Figure 1 illustrates how Layer 3 and Layer 4A of Component 1 sit within the group's layered pipeline and where the headline and body streams are reunited.

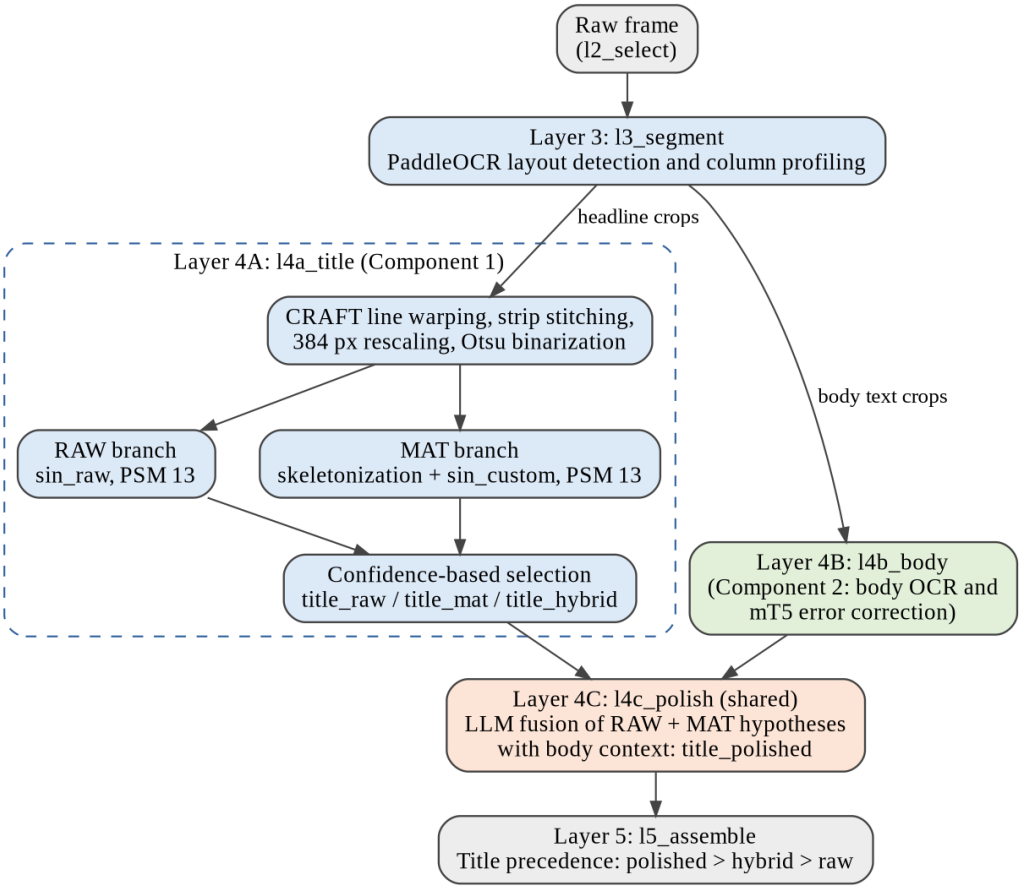


Figure 1: Processing flow of Component 1 within the complete solution

Source: Author. Blue: Component 1; green: Component 2; orange: shared layer; grey: frame selection and assembly.

4.3 Design Alternatives and Decisions

The recognition engine underwent three major architectural iterations before arriving at the production design:

- *Attempt 1 - Character-Level CNN Classification (ResNet-34 [20] / EfficientNet-V2 [23]):*
  - *Initial Rationale:* Sinhala characters possess distinct topological curvatures and cursive loops. Feeding cleanly segmented, skeletonized character glyphs into strong spatial feature extractors (ResNet-34 or EfficientNet-V2) was hypothesized to yield robust classification.

  - *Failure Mode (Segmentation Bottleneck):* The approach failed due to Sinhala script mechanics rather than CNN representation limits. Sinhala features touching conjuncts (*bandi akuru*) and complex vowel signs (*pili*) that wrap around, connect to, or sit above/below the base consonant. Slicing continuous cursive text into isolated bounding boxes proved brittle; any clipping of a modifier corrupted the glyph, resulting in model hallucinations. Furthermore, treating every base consonant and modifier combination as a discrete class inflated the output space to over 700 classes, requiring an intractable number of balanced character-level training crops.

- *Attempt 2 - End-to-End Vision Transformer (TrOCR [16]):*
  - *Initial Rationale:* TrOCR (microsoft/trocr-base-printed) uses a Vision Transformer (DeiT) encoder coupled with a text Transformer decoder, eliminating the need for character-level segmentation by transcribing entire text lines end-to-end.
  - *Failure Mode (Tokenization and Compute Overhead):* TrOCR's pre-trained Byte-Pair Encoding (BPE) tokenizers are optimized for Latin scripts and fragmented Sinhala Unicode sequences. Because Sinhala heavily relies on Zero Width Joiners (ZWJ) and strict modifier sequencing, the model produced disjointed Unicode codepoints that failed rendering. Additionally, fine-tuning Vision Transformers exceeded available GPU VRAM limits (Google Colab T4) and introduced excessive inference latency, making local edge deployment on an assistive device infeasible. This was a one-time, cloud-based trial on a 10,000-image subset (9,000 training / 1,000 testing, random seed 42, recorded in split_record.json); it was never implemented in the final system and is reported only to justify the rejection of end-to-end vision transformers.

- *Production Engine - Sequence-to-Sequence Recurrent Network (Tesseract LSTM OEM 1):*

  - *Design Resolution:* Tesseract v5's Long Short-Term Memory (LSTM) engine [13] with Connectionist Temporal Classification (CTC) loss [22] provided the optimal solution. CTC scans word images as continuous visual time-series sequences, mapping features directly to Unicode text strings without requiring isolated character bounding boxes. This retained the end-to-end benefits of TrOCR while operating with minimal computational overhead, enabling execution on resource-constrained consumer CPUs and GPUs.

- *Preprocessing Strategy - Hybrid Routing vs. Wholesale Replacement:*
  - *Design Resolution:* Although MAT skeletonization normalizes font variations, benchmarking showed that unmodified binarized images (RAW) yield superior accuracy and far lower latency on standard fonts (Section 6). Rather than replacing RAW preprocessing entirely, a hybrid routing design was implemented: the RAW and MAT pipelines run concurrently, and the output with the higher Tesseract confidence is selected (Section 4.4). Edge-density analysis, originally considered as a routing signal, is retained only for visualization. The combined accuracy and latency of this hybrid router have not yet been benchmarked, so it is reported as an implemented production feature and architectural prototype rather than a validated result.

4.4 Detailed Design

Component 1 is architected as two tightly integrated execution layers:

- **Layer 3 (l3_segment - Article Segmentation & Layout Analysis):**
  - *Macro Layout Detection (article_splitter_p1.py):* Uses PaddleOCR (PP-DocLayout_plus-L / PPStructureV3) to identify layout bounding boxes across the page, classify block types (title, text, image), and perform perspective unwarping. PaddleOCR operates script-agnostically, recognizing spatial text clusters regardless of Sinhala linguistic properties.
  - *Safety Buffering:* A dynamic margin buffer is added around all detected headline bounding boxes before cropping, ensuring no exterior vowel

signs (*combowa*, *al-lakuna*) are truncated during initial layout segmentation.

- *Close-Up Fallback Profiling (closeup.py, layout.py):* For zoomed-in camera framing where document borders are obscured, the system bypasses complex layout models and evaluates vertical/horizontal ink projection profiles and line heights to isolate headline bands (headline_bands()).
- *Line & Polygon Detection (article_splitter_p2.py):* CRAFT TextDetection [17] identifies precise word and character density maps within the headline regions, generating tight polygon coordinates that prevent inadvertent splitting of attached diacritics.

- **Layer 4A (l4a_title - Headline Preprocessing & Recognition):**
  - *Stage 1: Line Stitching (title_extractor.py, stage 1):* Takes CRAFT polygon crops, applies perspective warping to rectify curved text lines, normalizes each line segment, and horizontally stitches multi-line headlines into a uniform sequence strip.
  - *Stage 2: Scale Normalization & Binarization (title_extractor.py, stage 2):* Rescales the stitched strip to a fixed height of 384 pixels using Lanczos/Cubic interpolation while preserving the aspect ratio. It converts the image to grayscale and applies Otsu thresholding with polarity correction to guarantee clean black glyphs on a pure white background.
  - *Stage 3: MAT Skeletonization (title_extractor.py, stage 3):* Computes the Euclidean Distance Transform followed by Zhang-Suen thinning via skimage.morphology to reduce strokes to a 1-pixel-wide medial axis. It then applies morphological dilation using a circular footprint to expand strokes to a uniform 3-pixel width, preventing structural erosion during downstream neural pooling. The production mat_process also calls trace_yellow_lines, an iterative Python routine that traces rays along the inward gradient of the distance field.

- *Stage 4: Dual-Engine Recognition Helper (title_extractor.py, stage 4, title.py):* Executes OCR via custom Tesseract configurations:
- sin_raw.traineddata: Ingests RAW binarized crops under Page Segmentation Mode (PSM) 13 (raw line).
- sin_custom.traineddata: Ingests MAT-skeletonized images under PSM 8 (single word) in the benchmark evaluation and PSM 13 (raw line) on stitched strips in production.
- Hybrid routing (title.py): the RAW pipeline (sin_raw) and the MAT pipeline (sin_custom on mat_process skeletonized strokes) execute concurrently in parallel threads. Tesseract is invoked with -c tessedit_create_tsv=1, and the average per-word confidence (0-100) is computed over valid, non-empty words (rows flagged -1 are ignored); if TSV parsing fails but plain text is produced, a default confidence of 50.0 is assigned. The router selects the MAT output only if conf_mat > conf_raw; the strict inequality deliberately favors RAW on ties to avoid over-skeletonization of thin dependent vowel modifiers (*pilla*). If one model returns empty text, the non-empty hypothesis is used.
- The layer returns article.title_raw, article.title_mat and article.title_hybrid (the selected string). Geometric edge density is no longer used for routing.

4.5 Standards and Professional Practice

- **Character Encoding Standards:** Strict adherence to the Unicode Standard for the Sinhala Script (Unicode block U+0D80–U+0DFF), ensuring correct handling of the Zero Width Joiner (U+200D) required for conjunct consonants (*bandi akuru*) and touching ligatures.
- **Software Modularity & Clean Architecture:** Implemented in Python following PEP 8 coding conventions, maintaining explicit schema contracts (Article, Region) across system layers.
- **Fault-Tolerant Microservice Isolation:** Visual processing layers interface through decoupled execution scripts, preventing layout detection anomalies from terminating downstream text-to-speech services.

- **Assistive System Design Guidelines:** Adheres to accessibility standards emphasizing low latency, predictable text-flow serialization, and transparent audio notifications during image capture.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

The development, training, and execution environments span local workstations and distributed client-server architecture:

- *Training Environment:* Early exploratory trials utilizing the TrOCR architecture were conducted on cloud-based Google Colab instances equipped with NVIDIA Tesla T4 GPUs (15 GB VRAM). However, the final production sequence-to-sequence Tesseract LSTM models (sin_custom and sin_raw) were trained entirely on a local AMD Ryzen 5 7235HS CPU, demonstrating the feasibility of local model generation without relying on heavy cloud GPU infrastructure. Both models were fine-tuned with tesstrain from the pre-trained Sinhala model (START_MODEL=sin) in a WSL environment, with recovery from checkpoints after environment crashes. sin_raw plateaued at 82,110 iterations (best evaluation BCER 7.505% at iteration 79,965), and sin_custom ran to 1,056,361 iterations (best evaluation BCER 10.558% at iteration 1,044,490).
- *Core Machine Learning & Vision Libraries:*
  - **Python 3.10+** serves as the core runtime environment.
  - **PaddleOCR (PP-DocLayout_plus-L / PPStructureV3):** Provides macro-level document structure analysis, unwarping, and layout bounding-box segmentation.
  - **CRAFT TextDetection (craft_text_detector):** Generates micro-level character and word affinity heatmaps to yield precise polygon bounding boxes around text lines.
  - **OpenCV (cv2) & Pillow (PIL):** Executes geometric transformations, perspective unwarping, interpolation (Lanczos, Cubic, and Nearest Neighbor), and adaptive Otsu binarization.

  - **Scikit-Image (skimage.morphology):** Computes the Euclidean Distance Transform, Zhang-Suen thinning (skeletonize), and morphological circular dilation (disk(1)).
  - **Tesseract OCR v5:** Operates the neural sequence-to-sequence recognition engine via OEM 1 (LSTM-only mode).
- *Client-Side Mobile Stack:* A native, lightweight (~8 MB) Android thin-client application communicating over local HTTP protocols. Capture audio feedback is handled via Android's built-in TextToSpeech API, where GuidanceSpeaker chunks prompts into ~300-character utterances, manages queue prioritization (QUEUE_FLUSH for immediate alerts, QUEUE_ADD for sequential prompts), and monitors playback states with an UtteranceProgressListener.
- *Edge / Server Deployment Specifications:*
  - *Minimum Specification (CPU Mode):* Intel Core i5 (8th Gen+) or AMD Ryzen 5, 8 GB RAM, 5-10 GB available storage. Accommodates CPU-only inference with higher latency (~several seconds per frame).
  - *Recommended Specification (Accelerated Interactive Mode):* Intel Core i7 or AMD Ryzen 7, 16 GB RAM, dedicated NVIDIA GPU (GTX 1650, RTX 3050, RTX 4060 or higher with 4 GB–6 GB+ VRAM), SSD storage. Relies on CUDA support (torch.cuda.is_available()) to maintain sub-second layout segmentation. The MAT skeletonization stage is CPU-bound and is not accelerated by CUDA (Section 6.2).

5.2 Component Implementation

Component 1 is implemented across two distinct structural layers within the application repository (system/layers/):

Table A.1 in Appendix A maps each processing stage described below to the source module that implements it.

- **Layer 3 (l3_segment- Article Segmentation & Layout Analysis):**

  - article_splitter_p1.py *(Layout Detection)*: Executes PaddleOCR PPStructure to parse the entire document page and split it into distinct *articles* first. Once the individual articles are isolated, the model further parses each article to classify and separate its internal sub-regions (title, text, image). To eliminate boundary clipping during this macro-segmentation, an automated safety buffer is dynamically appended around all detected headline regions.
  - closeup.py & layout.py *(Framing Fallback)*: When unconstrained close-up camera framing obscures outer document borders, the system bypasses complex layout models and evaluates vertical and horizontal ink projection profiles and font line-height differentials (headline_bands(), headline_for_block()) to isolate headline strips within the visible article.
  - article_splitter_p2.py *(Line & Polygon Extraction)*: Applies CRAFT TextDetection across isolated blocks to generate tight polygon boundaries for text lines, cleanly separating headline segments from body paragraphs prior to downstream handoffs.
- **Layer 4A (l4a_title - Headline Extraction and OCR):**
  - title_extractor.py, stage 1 *(Sentence Strip Stitching)*: Ingests CRAFT polygon crops, performs perspective warping (perspective_warp) to flatten skew, normalizes heights, and stitches multi-line titles into unified horizontal image strips.
  - title_extractor.py, stage 2 *(Scale Normalization & Binarization)*: Rescales stitched title strips strictly to a target height of 384 pixels using Lanczos/Cubic interpolation while preserving the natural aspect ratio. It performs grayscale conversion, Otsu global thresholding, polarity detection (inverting dark-on-light to white background with black ink), and filters isolated speckle noise.
  - title_extractor.py, stage 3 *(MAT Skeletonization)*: Implements Medial Axis Transform based thinning via the Zhang-Suen algorithm (skimage.morphology.skeletonize), collapsing strokes into a 1-pixel-wide

medial axis skeleton. It then executes morphological dilation with a disk(1) structuring element to expand the stroke to a standardized 3-pixel width, safeguarding delicate vowel signs (*pili*) against neural downsampling artifacts.

- title_extractor.py, stage 4 & title.py *(OCR Execution & Adjudication)*: Interfaces with the local Tesseract engine via custom model weights:
- sin_raw.traineddata: Evaluates unmodified binarized strips using Page Segmentation Mode (PSM) 13 (raw line).
- sin_custom.traineddata: Evaluates MAT-skeletonized images using PSM 8 (single word) in the benchmark evaluation and PSM 13 (raw line) on stitched strips in production.
- Generates article.title_raw (RAW baseline transcription), article.title_mat (MAT transcription) and article.title_hybrid (the string selected by comparing average Tesseract word confidences, with ties resolved in favor of RAW).

5.3 Integration with the Other Components

Component 1 interfaces directly with the other components in Group R26-DS-002 through modular contracts and shared object schemas:

- *Upstream Ingestion:* Receives the stabilized document frame from Layer 2 (l2_select), which filters camera burst buffers for minimum blur and optimal contrast.
- *Bifurcated Routing Handoff:* Layer 3 (l3_segment) acts as the architectural splitter. It first isolates distinct articles on the page, then splits each article into its constituent title and body elements.
  - The body text paragraph crops are routed to **Component 2** (l4b_body), where they undergo baseline OCR and context-aware error correction via a fine-tuned mT5-Small transformer model.

  - The headline regions are retained internally and processed by Layer 4A (l4a_title).

- *Assembly Unification:* The extracted headline hypotheses (title_raw, title_mat, title_hybrid) are reconciled with body-text context in **Layer 4C (l4c_polish)** to give title_polished; the headline (precedence title_polished > title_hybrid > title_raw) and the corrected body paragraphs then reunite at **Layer 5 (l5_assemble)**, which aggregates them into structured Article objects that preserve document reading order.

- *Downstream NLP Handoff:* The assembled document structure is indexed into a ChromaDB vector database in **Component 3 (l6_generator)** for Retrieval-Augmented Generation (RAG) and question answering.

- *Voice & Personalization Decoupling:* The pipeline interfaces with **Component 4 (l7_personalization)**, which runs intent detection (NLLB-600M + Llama 3.2 1B via Ollama) and River online learning (Softmax regression) to adapt output styles (Simple, Detailed, Step-by-Step). To prevent dependency conflicts between disparate machine learning libraries, Component 4 operates as an independent FastAPI microservice (svc-voice), communicating over HTTP while Component 1 marks local stubs (source: "stub") to prevent unverified data routing.

Figure 2 summarizes how Component 1 interfaces with the other components of Group R26-DS-002.

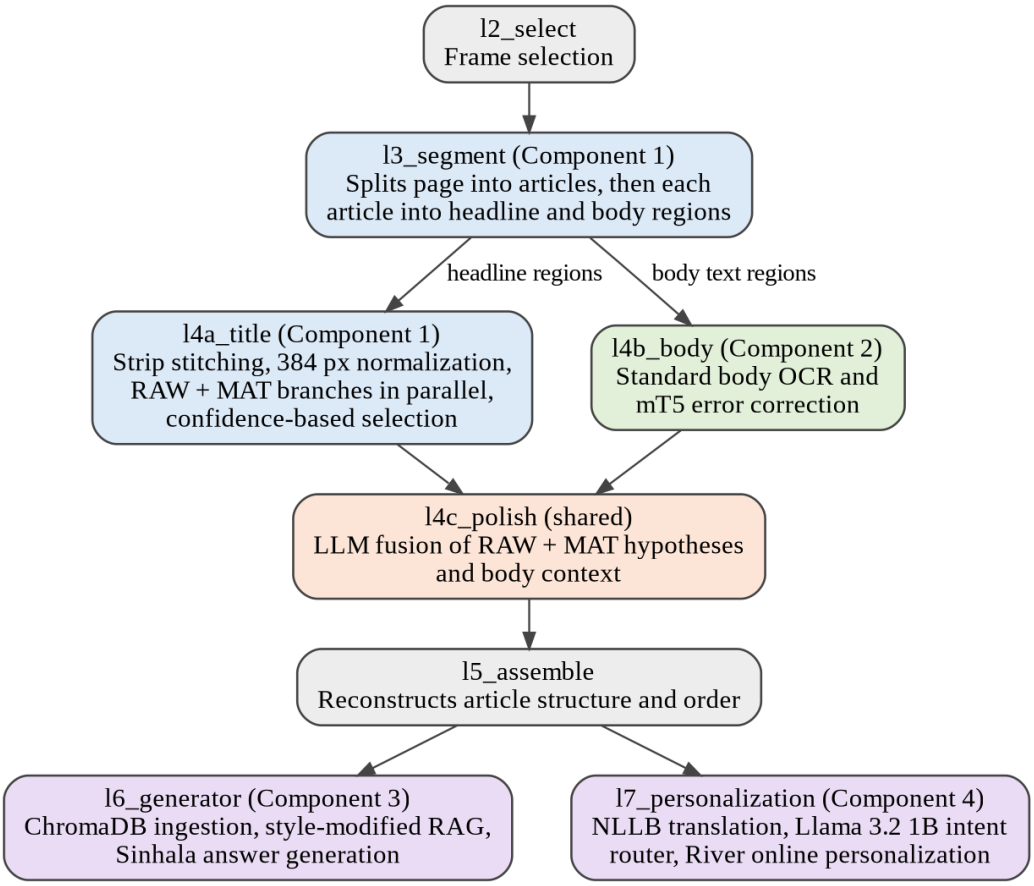


Figure 2: Integration of Component 1 with the other components of Group

Source: Author. Blue: Component 1; green: Component 2; orange: shared layer; purple: Components 3 and 4

5.4 Verification and Software Testing

Verification procedures combined algorithmic benchmarking, pipeline assertion checks, and developer-simulated user testing:

- *Dataset-Level Regression Verification:* The pipeline was systematically validated against the 8,872-image common evaluation subset of the held-out 10,000-image SinOCR test set and the 10,000-image synthetic 135-font adversarial dataset to verify that Character Error Rate (CER) calculations via Levenshtein edit-distance remained deterministic and repeatable.

- *Bounding Box & Segmentation Verification:* Visual and coordinate assertion testing verified that the safety margin padding in article_splitter_p1.py and the density-based clustering in CRAFT completely eliminated diacritic clipping on complex conjuncts (*bandi akuru*) and vowel modifiers (*combowa*, *keti is-pilla*).
- *Degradation & Stub Verification:* The voice-routing layer (layers/l0_voice/voice.py) and HTTP microservice endpoints were tested under simulated service outages. When external NLP endpoints timed out, the local literal router successfully handled core commands (STOP, NEXT, PREVIOUS, REPEAT) and properly stamped fallback outputs as source: "stub" to avoid reporting unverified model outputs.
- *Simulated Assistive User Testing:* Because formal user testing with visually impaired individuals is designated for future deployment phases, operational verification was conducted via developer simulation. Evaluators executed end-to-end reading workflows with eyes closed, relying entirely on Android GuidanceSpeaker audio prompts ("a little closer", "hold steady") to position documents, trigger capture, and navigate extracted articles via voice keywords.

5.5 Deployment and Operation

- *Client Deployment:* The Android client is compiled as a minimal, standalone package requiring standard camera and microphone permissions. It offloads all heavy vision and neural processing to the backend server, maintaining minimal battery drain and device storage footprints (~8 MB).
- *Server Operation & Ephemeral Execution:* The backend execution pipeline operates either on a dedicated cloud server or on a local personal computer acting as an edge server within a domestic or institutional local area network (LAN) (e.g., schools for the visually impaired).
- *Memory Management and Privacy:* Incoming image frames, intermediate CRAFT polygon crops, and binarized strips are processed exclusively in volatile memory buffers. Post-OCR synthesis, all raw and intermediate visual representations are explicitly dereferenced and purged by garbage collection, ensuring that private or sensitive documents are not retained on storage disks.

6 RESULTS AND VALIDATION

6.1 Research Results

The research results were derived from evaluating the RAW and MAT pipelines across both in-distribution datasets and adversarial stress tests:

Table 6.1 to Table 6.3 summarize the principal quantitative results, which are described in detail below.

Table 2:Baseline evaluation on the common 8,872-image SinOCR subset

Measure	RAW	MAT
Mean CER	0.0586	0.0922
Images with strictly lower CER	4,428	1,584
Tied images	2,860	2,860
Zero-error predictions	2,907	1,977

Source: Author's experiments. Paired two-tailed t-test on per-image CER: $p = 1.655 \times 10^{-70}$.

Table 3:Adversarial stress test results (135 fonts, 10,000 images)

Measure	RAW	MAT
Fonts with strictly lower CER (of 135)	105	29
Tied fonts	1 (apex33Copy)	1 (apex33Copy)
Zero-error predictions (of 10,000)	524	379

Source: Author's experiments.

Table 4:Geometric edge-density complexity of the font groups

Font group	Fonts (n)	Mean complexity	SD
MAT-favored fonts	29	1.59	0.60
RAW-favored fonts	105	1.25	0.45

Source: Author's analysis. Welch two-sample t-test: $t(37.2) = 2.87$, $p = 0.0067$.

- *Baseline Evaluation (SinOCR):* On the common 8,872-image evaluation subset, the RAW pipeline established itself as the stronger generalist, achieving a mean Character Error Rate (CER) of 0.0586 compared to 0.0922 for the MAT pipeline. RAW produced a strictly lower CER on 4,428 images, MAT won on

1,584 images, and the two pipelines tied on the remaining 2,860 images (4,428 + 1,584 + 2,860 = 8,872). Of these ties, 1,522 (53.2%) are images that both pipelines transcribed perfectly (CER = 0) and 1,338 (46.8%) are identical non-zero CERs (mean 0.0637). Across the whole subset, RAW produced 2,907 zero-error predictions and MAT 1,977. A paired, two-tailed t-test confirmed this difference as highly significant ($p = 1.655 \times 10^{-70}$).

- *Adversarial Stress Test:* On the 10,000-image, 135-font synthetic stress test, RAW produced a strictly lower CER on 105 fonts, MAT led on 29 fonts, and one font (apex33Copy) was tied (105 + 29 + 1 = 135). RAW also yielded more perfect (zero-error) predictions overall (524 vs. 379).
- *Geometric Edge Density Analysis:* To objectively measure why MAT excelled on those specific 29 fonts, a geometric complexity score was computed for each font (Canny edge pixels as a percentage of all pixels in an Otsu-binarized sample glyph image). The 29 MAT-favored fonts had a mean complexity score of 1.59 (SD 0.60), compared to 1.25 (SD 0.45) for the 105 RAW-favored fonts, a 27.4% higher mean edge density. A Welch two-sample t-test showed this difference to be statistically significant ($t(37.2) = 2.87$, $p = 0.0067$), consistent with MAT's accuracy advantage being concentrated in structurally complex typography.

6.2 Product and Technical Results

The technical viability of the system was measured via end-to-end processing latency to ensure it meets the real-time requirements of an assistive device:

Table 6.4 summarises the measured processing latency on the benchmark crops and in the production pipeline.

Table 6.4: Processing latency on the AMD Ryzen 5 7235HS CPU

Table 5:Processing latency on the AMD Ryzen 5 7235HS CPU

Setting	RAW	MAT
Preprocessing on benchmark crops (ms per crop)	~1.2	~29
Production stitched strips, total (ms per crop)	~213	~5,777

Setting	RAW	MAT
of which MAT preprocessing (mat_process)	Not reported	~5,622
of which Tesseract execution	Not reported	~155

Source: Author's benchmarks.

- *Computational Cost:* Latency was benchmarked locally on the AMD Ryzen 5 7235HS CPU and must be read in two settings. On the same pre-segmented benchmark crops, preprocessing took about 1.2 ms per crop for RAW and about 29 ms for MAT. In the production pipeline, headline words are stitched into one horizontal strip rescaled to 384 pixels in height (often more than 2,000 pixels wide, i.e. over a million pixels), and the standard RAW model averaged ~213 ms per crop whereas the MAT model averaged ~5,777 ms per crop (about 27x). For MAT, ~5,622 ms is spent in MAT preprocessing (mat_process) and ~155 ms in Tesseract execution, so the overhead comes from preprocessing rather than OCR inference. Profiling mat_process on a synthetic 384 x 2,500-pixel binarized text strip (not a real headline) showed that the standard morphology steps are cheap (distance transform ~5-50 ms, Zhang-Suen thinning ~11 ms, disk(1) dilation ~7 ms), whereas the iterative Python routine trace_yellow_lines, which traces rays along the inward distance gradient, accounted for nearly all of the time (~11.6 s on that strip, on the same CPU; the absolute figure differs from the production measurement because the strip was synthetic). The latency penalty is therefore attributable to this routine in the current implementation, not to Medial Axis skeletonization itself. Earlier end-to-end figures of ~1,169 ms (RAW) vs ~1,197 ms (MAT), which included TrOCR-phase inference of ~1,168 ms, are superseded and are not part of the final results.

- *Architectural Impact:* This overhead rules out applying the current MAT implementation uniformly to stitched production strips, although, because most of the cost sits in one routine, it is likely reducible by optimization (not tested). The implemented hybrid router runs both branches concurrently, so its wall-clock latency is expected to be bounded by the slower MAT branch (~5.8 s per

crop on the test CPU; not measured), before any Layer 4C processing. The hybrid is therefore an implemented production feature and architectural prototype; its combined accuracy and latency were not benchmarked.

6.3 User or Stakeholder Validation

Validation was conducted across two distinct domains: perceptual visual difficulty and system usability:

- *Perceptual Survey:* To determine if MAT's algorithmic advantage aligns with human-perceived difficulty, a survey was shared with members of the general public. Each trial showed two images, one from a font where MAT excelled and one from a font where RAW excelled, and asked which looked more complex and harder to recognize; raters were not told which image was which. Left/right placement was randomized for every trial, and a participant log ensured that no evaluator saw the same pair twice. Twenty raters completed all 20 comparisons and one rater stopped after 4, giving 404 trials (20 x 20 + 4), each a unique image pair seen exactly once (808 image presentations). Raters were sighted volunteers, not visually impaired users. Raters chose the MAT-favored image as the harder one in 54.5% (220 of 404) of trials. An exact two-sided binomial test against 50% gives $p = 0.0815$, which does not reach the conventional 0.05 threshold, so the result is a suggestive trend rather than conclusive evidence. Trials from the same rater are not fully independent, which the binomial test does not account for.

- *Usability Validation:* System-level validation of the Android audio cues (GuidanceSpeaker) and voice-command routing was conducted via developer simulation. Developers operated the system under simulated visual impairment (eyes closed), successfully relying on real-time audio feedback to position the camera, capture documents, and navigate the parsed articles. Formal usability testing with actual visually impaired stakeholders is marked as a critical future validation phase.

6.4 Integration Results

Component 1 successfully fulfilled its contract within the broader Group R26-DS-002 architecture. The l3_segment layout detection accurately separated headline and body regions without clipping critical Sinhala vowel modifiers. The bifurcated handoff proved stable: headlines were cleanly processed by the RAW/MAT OCR pipelines in l4a_title, while body text was simultaneously processed and corrected by the mT5 model in l4b_body (Component 2). Both streams were successfully reunited in sequential reading order at l5_assemble, confirming that the multi-layer pipeline's synchronization barrier reliably reunites structured text for the downstream RAG and voice generation modules. Layer 5 waits for both the headline and body streams to resolve before building the Article object, and state is passed through schema objects rather than shared global variables. Developer trials on multiple newspaper captures were repeated until observed errors were resolved and end-to-end behavior (headline audio followed by the matching body text) was satisfactory; no fixed number of trials and no dedicated high-throughput load testing were recorded.

6.5 Summary of Findings

The empirical evaluations demonstrate that while the standard RAW pipeline remains the stronger and significantly faster generalist for Sinhala OCR (CER 0.0586 vs 0.0922; ~213 ms vs ~5,777 ms per crop), Medial Axis Transform (MAT) skeletonization is a highly valuable specialist technique. MAT successfully normalizes structural variation in geometrically complex, heavily stylized Sinhala fonts. Because of the high computational cost of the current implementation (~5,777 ms vs ~213 ms), MAT cannot be deployed uniformly. Its product value is as a complementary branch in a confidence-based hybrid router intended to catch failures on difficult typographic crops; that hybrid is implemented but its combined accuracy and latency remain to be benchmarked.

7 DISCUSSION

7.1 Interpretation of Findings

The empirical results demonstrate that while the RAW pipeline remains the stronger and significantly faster generalist for standard Sinhala text, the MAT pipeline possesses a distinct advantage on geometrically dense, structurally complex typography. The geometric edge-density analysis confirms that MAT's success is not random but objectively tied to high intrinsic font complexity (mean edge density of 1.59 for MAT-won fonts vs. 1.25 for RAW-won fonts). Furthermore, the human perceptual survey where 54.5% of MAT-won images were judged as visually harder suggests a possible trend ($p = 0.0815$, not significant) that MAT's algorithmic advantage aligns with human-perceived visual difficulty.

7.2 Comparison with Literature Baselines and Existing Solutions

Contemporary Sinhala OCR research and commercial baseline models have almost exclusively relied on raw binarized pixel images fed into end-to-end deep learning architectures. The broader OCR field largely abandoned topological preprocessing methodologies when transitioning to neural feature learning. By comparing RAW and MAT pipelines on the same LSTM engine, this research demonstrates that skeletal representations still provide measurable, specialized value for structurally complex scripts like Sinhala, challenging the assumption that modern neural engines automatically capture all necessary topological signals from raw pixels.

7.3 Research Contribution

This study, to the best of our knowledge provides the first applied empirical evaluation of Medial Axis Transform (MAT) skeletonization as a preprocessing stage ahead of a modern LSTM-based OCR engine designed specifically for the Sinhala script. Earlier Sinhala OCR work, and the commercial and open-source baselines it is compared against, has largely fed binarized pixel images directly into end-to-end learning models. Skeletal representations were rarely tested in this setting, and almost never under controlled conditions on the same recognition engine. This research fills that gap with a like-for-like comparison. The RAW and MAT pipelines share one LSTM

engine, so any performance difference can be attributed to the input representation rather than to the model.

The contribution is made up of four connected parts.

1. **A four-stage validation methodology.** The study combines four kinds of evidence that are rarely brought together in a single OCR evaluation:
    - quantitative benchmark testing of the two pipelines;
    - a human perceptual survey on the visual difficulty of the test images;
    - adversarial stress testing on complex fonts;
    - an objective geometric complexity analysis based on edge density.

Each stage checks the others, so a conclusion does not depend on accuracy scores alone. The methodology can be reused to evaluate other preprocessing techniques on Sinhala or on other complex scripts.

2. **An empirical account of when each pipeline wins.** The results show that the RAW pipeline remains the stronger and significantly faster generalist on standard Sinhala text. The MAT pipeline has a distinct advantage on geometrically dense, structurally complex typography. This replaces a simple "which is better" question with a more useful one: which input representation suits which kind of font.

3. **A quantitative explanation of MAT's advantage.** The geometric edge-density analysis links MAT's wins to intrinsic font complexity. The mean edge density was 1.59 for MAT-won fonts and 1.25 for RAW-won fonts. This indicates that MAT's success is systematic and not random variation($t(37.2) = 2.87$, $p = 0.0067$, 29 versus 105 fonts)

4. **Evidence on the link to human perception.** In the human perceptual survey, 54.5% of the images on which MAT won were judged visually harder. This points toward an alignment between MAT's algorithmic advantage and human-perceived difficulty, but the result is not statistically significant ($p = 0.0815$). We therefore report it as a suggestive trend that needs a larger sample, not as a

confirmed effect. There were 21 survey participants with 404 trails and 808 images rated.

Together, these findings challenge the assumption that modern neural engines automatically capture all useful topological information from raw pixels. For structurally complex scripts such as Sinhala, an explicit skeletal representation can still add measurable, specialized value. The findings also suggest a practical direction: a selective or hybrid strategy that chooses MAT or RAW input according to measured font complexity, rather than replacing one with the other. Such a strategy would use MAT where it helps and keep RAW's speed advantage elsewhere. This idea has not been tested in this study (see Section 7.5 and Future Directions).

7.4 Product Contribution

Component 1 delivers a robust, deployable visual ingestion and headline extraction module that handles the complexities of Sinhala document layouts. It is the entry point of the assistive reader ecosystem developed by Group R26-DS-002. It converts newspaper pages and stylized documents into clean, complete headline text that the downstream components can use.

Problem addressed. Sinhala headlines are difficult for conventional OCR pipelines. They are set in large, decorative, or stylized typefaces, and their vowel diacritics (pili) can be placed to the left, right, above, or below the base consonant. When a text region is cropped by a simple bounding box, these multi-positional marks are often cut off. The OCR then returns wrong or incomplete words, and the error carries through every later stage.

Solution design. The module combines three techniques:

- **PaddleOCR** for robust text detection and recognition;
- **CRAFT text-line focusing** to localize text at character level and keep complete text lines, including their diacritics, inside the crop;
- **a hybrid OCR architecture** that combines the strengths of the individual engines and reduces the failures of any single one.

Together, these prevent the mechanical clipping of pili and allow the module to read complete newspaper pages and stylized documents without catastrophic failures on headline typography. The headline extraction accuracy, character or word error rate, and processing time per page can be later measured in fine tuning of the application.

Contribution to the integrated solution. The module is the first stage of the pipeline. Its output feeds the later components of the assistive reader, so its accuracy sets the upper limit for the whole system. Reliable headline extraction means that later processing starts from correct text and not from corrupted input. The Headline along with the body from the Component 2 is passed onto Component 3, and while headline may contain significantly less information than the body, it carries a significant weight.

Practical value. For users of the assistive reader, who may rely on it to access printed news, this means:

- reliable access to headlines, which are often the key to deciding what to read;
- support for real newspaper layouts and not only clean, single-column text;
- a module built for Sinhala-specific script challenges, not a generic OCR tool applied to Sinhala.

Limitations. The module's performance depends on image quality and layout complexity. Aside from environmental and hardware limitations such as low-resolution scans, environmental lighting or curved text (which are not within the module's control), there are other limits which are discussed in Section 7.5.

7.5 Limitations and Threats to Validity

Several limitations restrict the generalizability of the findings:

- *Synthetic Adversarial Data:* The 135-font stress test is entirely synthetic; therefore, the demonstrated MAT advantages do not automatically confirm identical performance gains on real-world noisy, heavily degraded, or handwritten Sinhala documents.
- *Statistical Thresholds:* The human perceptual survey results (exact two-sided binomial $p = 0.0815$) did not reach the conventional 0.05 threshold for

statistical significance, meaning the correlation between MAT's performance and human-perceived difficulty is suggestive rather than conclusive.

- *Aggregate Metrics vs. Character-Level Confusion:* Because the Tesseract LSTM engine processes continuous text sequences rather than isolated characters, the study reports aggregate Character Error Rates (CER). This obscures granular character-level confusion analysis, making it impossible to identify exactly which specific Sinhala conjuncts (*bandi akuru*) or diacritics drive MAT's accuracy advantage.
- *Pipeline Confounds:* In the benchmark evaluation, RAW and MAT differ not only in preprocessing but also in the trained model and the Tesseract PSM (13 vs 8), and the two training runs ended at very different iteration counts (82,110 for sin_raw vs 1,056,361 for sin_custom); sin_custom still ended with the higher BCER. Observed CER differences therefore cannot be attributed to skeletonization alone. The PSM rationale for MAT (preventing skeleton-induced word fragmentation) has not been tested by re-running both pipelines under the same PSM.
- *Benchmark-to-Production Gap:* Accuracy was measured on pre-segmented word or short-phrase crops, whereas production feeds stitched multi-word strips and runs both models with PSM 13. Production accuracy, including whether MAT under PSM 13 suffers the word fragmentation that PSM 8 was meant to prevent, was not evaluated.
- *Hybrid Router and Layer 4C Not Evaluated:* The evaluation targets the individual RAW and MAT models. The combined output is produced by the confidence-based router and the shared Layer 4C LLM fusion, which fall outside this component's evaluation, so the hybrid is reported as a prototype. Tesseract confidence scores come from two separately trained models and may not be calibrated against each other.
- *Stress-Test Font Selection:* The 135 fonts were hand-selected by the author for visual complexity; overlap with the 200 SinOCR fonts cannot be excluded.

- *Survey Design:* The survey used 21 volunteers (404 unique pairs; 20 trials per rater except one rater with 4). Trials within a rater are not independent, and the survey measures perceived complexity, not recognition performance of visually impaired users.
- *Latency Measurement:* Latency was measured on a single CPU (AMD Ryzen 5 7235HS); MAT cost is dominated by the iterative trace_yellow_lines routine inside mat_process on large stitched strips (profiled on a synthetic strip, not on real headlines).

7.6 Implications

The contrast between MAT's advantage on complex fonts and its severe computational cost (~5,777 ms vs ~213 ms per crop, dominated by the trace_yellow_lines routine used to visually draw MAT skeletons rather than by skeletonization itself) has direct architectural implications for assistive software design. It motivates confidence-based routing rather than a uniform preprocessing strategy, with RAW as the default output. The current implementation runs both branches in parallel and selects the more confident result, which preserves accuracy options but leaves wall-clock latency bounded by MAT; running MAT only when RAW confidence is low, or optimizing the trace_yellow_lines routine, are the obvious optimizations and are listed as future work.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value Proposition and Target Users

The primary value proposition of this system is providing script-aware, highly accurate assistive reading capabilities to visually impaired and elderly Sinhala-speaking individuals, a demographic heavily reliant on printed materials for daily information. By extracting complex Sinhala typography with high accuracy, the system grants these individuals autonomous access to newspapers, textbooks, and formal documents. Secondary target users include OCR developers and research institutions seeking font-invariant Sinhala recognition engines and context-aware error correction pipelines. Tertiary stakeholders include Sri Lankan disability advocacy groups and government education agencies looking for affordable, open-source assistive infrastructure.

8.2 Market and Competitive Position

Currently available commercial assistive technologies, such as JAWS and Google Lookout, are predominantly English-centric and fail to provide functional support for the complex Sinhala script. Furthermore, early project iterations considered custom-built smart glasses, but this approach was abandoned as specialized hardware is cost-prohibitive for low-resource demographics and struggles with varying lighting and angles. Instead, the current system capitalizes on the ubiquitous smartphone market, utilizing heavily optimized, built-in smartphone camera pipelines (auto-focus, HDR) via a free, lightweight mobile application. This software-centric pivot drastically reduces the barrier to entry, positioning the solution favorably against expensive, proprietary assistive hardware.

8.3 Business or Sustainability Model

The sustainability model relies on a decentralized, open-source distribution strategy. The mobile client is a lightweight (~8 MB) thin-client application that can be freely downloaded and distributed. To manage the computational weight of the AI processing (Component 1's Tesseract models, Component 2's mT5, and Component 3's RAG), the backend can be hosted flexibly:

- *Centralized Hosting:* Subsidized cloud servers backed by NGOs, government grants, or university research clusters to provide free public access.
- *Localized Edge Hosting:* A localized desktop server application that can be installed on personal PCs or institutional machines (e.g., at schools for the blind or rehabilitation centers). This localized model transforms existing institutional computers into offline processing hubs, entirely eliminating ongoing cloud operational costs while ensuring data privacy.

8.4 Scalability Intellectual Property and Compliance

By segregating the computationally heavy vision and neural processing to a backend server (cloud or local edge), the mobile application scales effortlessly across low-end consumer smartphones. The local HTTP network protocols minimize bandwidth consumption and data usage. In terms of compliance and privacy, the ephemeral, zero-retention memory architecture ensures that images of sensitive personal documents are never stored or transmitted outside the designated processing loop. The foundational source code leverages open-source libraries (PaddleOCR, Tesseract, Llama), allowing for permissive open-source licensing that encourages further community-driven development and academic contribution.

8.5 Contribution to the Sustainable Development Goals

The deployment of this assistive Sinhala reader aligns directly with the United Nations Sustainable Development Goals (SDGs):

- **SDG 10: Reduced Inequalities:** The system directly addresses Target 10.2 (empowering the social, economic, and political inclusion of all, irrespective of disability) by eliminating the information barrier for visually impaired Sinhala speakers, promoting equal access to societal participation [3].
- **SDG 4: Quality Education:** The technology aligns with Target 4.5 (eliminating gender disparities in education and ensuring equal access to all levels of education and vocational training for the vulnerable, including persons with disabilities) by providing blind students with the autonomous ability to read printed Sinhala educational materials and textbooks [21].

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This research evaluated Medial Axis Transform (MAT) skeletonization as a topological preprocessing stage for Sinhala optical character recognition ahead of a modern LSTM engine. The findings demonstrate that the standard RAW pipeline is the more accurate and significantly faster default for general Sinhala typography (mean CER 0.0586 vs 0.0922 on 8,872 images), while MAT skeletonization normalizes structural variation and gives lower CER on a subset of structurally complex fonts (29 of 135 stress-test fonts). The central hypothesis (Section 1.6), that MAT significantly improves recognition accuracy, was therefore not supported in general and was supported only for those fonts. MAT is accordingly positioned as a specialist branch rather than a universal replacement. A confidence-based hybrid router that combines both branches is implemented, but its combined accuracy and latency have not yet been benchmarked.

9.2 Achievement of Objectives

The primary objective of benchmarking a topologically preprocessed (MAT) pipeline against a baseline (RAW) pipeline using a shared Tesseract LSTM architecture was successfully achieved. The rigorous four-stage validation methodology was completed in full, encompassing an 8,872-image held-out baseline evaluation, a 404-trial human perceptual survey, a 10,000-image 135-font adversarial stress test, and an objective algorithmic geometric complexity analysis. Furthermore, the component was successfully integrated into the overarching Group R26-DS-002 assistive text reader ecosystem, meeting all structural layout and text-handoff requirements.

9.3 Individual Research and Product Contributions

- **Research Contribution:** To the best of our knowledge, conducted the first empirical study evaluating MAT skeletonization as a preprocessing stage ahead of a modern, sequence-to-sequence LSTM OCR engine specifically designed for the Sinhala script.

- **Product Contribution:** Engineered the complete visual ingestion pipeline across Layer 3 (l3_segment) and Layer 4A (l4a_title). This included implementing PaddleOCR layout splitting with dynamic safety buffering, CRAFT polygon detection, curved line stitching, 384px height rescaling, Otsu thresholding, and MAT skeletonization with 3px dilation.
- **Model Generation:** Trained and integrated custom Tesseract LSTM models (sin_custom and sin_raw) capable of transcribing standard and skeletonized Sinhala text without clipping delicate diacritics (*pili*) or touching conjuncts (*bandi akuru*).

9.4 Future Directions

- *Character-Level Confusion Analysis:* Conduct a post-hoc analysis utilizing the edit-distance alignments generated during CER computation to pinpoint exactly which specific Sinhala ligatures, conjuncts, and vowel diacritics benefit most from MAT skeletonization.
- *Skeleton-Derived Complexity Metrics:* Develop an algorithmic metric based on branch-point or junction density directly on the medial axis to test whether skeletonization objectively reduces structural complexity, rather than only measuring intrinsic font complexity.
- *Real-World Document Validation:* Extend the empirical evaluation beyond synthetic datasets to test the hybrid pipeline on in-the-wild, degraded, and historical scanned Sinhala documents.
- *In-Situ Assistive User Trials:* Execute formal usability trials directly with visually impaired and blind participants in Sri Lanka to validate the effectiveness of the Android client's audio feedback and voice routing in real-world environments.
- *Joint Evaluation of Hybrid Routing and Polishing:* Together with the owners of the shared Layer 4C, evaluate the combined headline output, measure its end-to-end latency, and test sequential gating (run MAT only when RAW confidence is low) and optimization of the trace_yellow_lines routine.

- *Production-Condition Re-Evaluation:* Re-run RAW and MAT on stitched headline strips under the production PSM (13), and under a common PSM with matched training iterations, to remove the confounds listed in Section 7.5.

REFERENCES

[1] World Health Organization, "Vision impairment and blindness," Fact sheet, Feb. 2026. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment

[2] UNPRPD, "Situational Analysis of the Rights of Persons with Disabilities in Sri Lanka," Jan. 2025. [Online]. Available: https://unprpd.org/new/wp-content/uploads/2025/01/SriLankaSitAn.pdf

[3] United Nations, "Goal 10: Reduce inequality within and among countries," 2026. [Online]. Available: https://www.un.org/sustainabledevelopment/inequality/

[4] N. de Silva, "Survey on Publicly Available Sinhala Natural Language Processing Tools and Research," arXiv:1906.02358, 2019.

[5] I. Anuradha, C. Liyanage, and H. Wijayawardhana, "Deep Learning Based Sinhala Optical Character Recognition (OCR)," in Proc. 20th Int. Conf. Advances in ICT for Emerging Regions (ICTer), 2020.

[6] P. Velayuthan and T. D. Ambegoda, "Benchmarking OCR Models for Sinhala and Tamil Document Digitization," in Proc. ERU Symposium 2024, Engineering Research Unit, Univ. of Moratuwa, 2024, doi: 10.31705/ERU.2024.7.

[7] T. Madhusha, H. D. M. Bhagya, S. Godakanda, J. P. D. D. Semini, K. Pulasinghe, and P. K. P. G. Panduwawala, "Mobile Base Sinhala Book Reader for Visually Impaired Students," Int. Res. J. Innovations Eng. Technol. (IRJIET), 2023.

[8] K. Gunathilaka, D. Hewagama, S. Pushpakumara, and T. D. Ambegoda, "SinFUND and SinOCR: Benchmarks for Sinhala Handwritten OCR and Template-Free Form Understanding," Research Square, preprint, Sep. 2025, doi: 10.21203/rs.3.rs-6976719/v1.

[9] L. Lam, S.-W. Lee, and C. Y. Suen, "Thinning Methodologies - A Comprehensive Survey," IEEE Trans. Pattern Anal. Mach. Intell., vol. 14, no. 9, pp. 869-885, Sep. 1992.

[10] S. Bag and G. Harit, "A Medial Axis Based Thinning Strategy for Character Images," arXiv:1103.0738, Mar. 2011.

[11] S. Bag and G. Harit, "A Medial Axis Based Thinning Strategy and Structural Feature Extraction of Character Images," in Proc. IEEE Int. Conf. Image Processing (ICIP), 2010.

[12] S. Cheng, Y. Ren, and Y. Yang, "SSR-GNNs: Stroke-Based Sketch Representation with Graph Neural Networks," in Proc. IEEE/CVF Conf.

Computer Vision and Pattern Recognition Workshops (CVPRW), 2022, pp. 5131-5141, doi: 10.1109/CVPRW56347.2022.00561.

[13] R. Smith, "An Overview of the Tesseract OCR Engine," in Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR), IEEE, 2007, pp. 629-633.

[14] R. Prabha et al., "Vivoice -- Reading Assistant for the Blind using OCR and TTS," in Proc. 2022 Int. Conf. Computer Communication and Informatics (ICCCI), IEEE, 2022.

[15] K. Gunathilaka, D. Hewagama, S. Pushpakumara, and T. D. Ambegoda, "SinOCR and SinFUND - Sinhala OCR and Form Understanding Datasets," IEEE DataPort, 2024, doi: 10.21227/hhez-0r18.

[16] M. Li et al., "TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models," arXiv:2109.10282, 2021.

[17] Y. Baek, B. Lee, D. Han, S. Yun, and H. Lee, "Character Region Awareness for Text Detection," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2019, pp. 9365-9374.

[18] T. Y. Zhang and C. Y. Suen, "A Fast Parallel Algorithm for Thinning Digital Patterns," Commun. ACM, vol. 27, no. 3, pp. 236-239, Mar. 1984.

[19] N. Otsu, "A Threshold Selection Method from Gray-Level Histograms," IEEE Trans. Syst., Man, Cybern., vol. 9, no. 1, pp. 62-66, Jan. 1979.

[20] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770-778.

[21] United Nations, "Goal 4: Ensure inclusive and equitable quality education," 2026. [Online]. Available: https://www.un.org/sustainabledevelopment/education/

[22] A. Graves, S. Fernandez, F. Gomez, and J. Schmidhuber, "Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks," in Proc. 23rd Int. Conf. Machine Learning (ICML), 2006, pp. 369-376.

[23] M. Tan and Q. V. Le, "EfficientNetV2: Smaller Models and Faster Training," in Proc. 38th Int. Conf. Machine Learning (ICML), 2021.

GLOSSARY

Bandi akuru – Sinhala touching or stacked conjunct consonants, in which two consonants are joined into a single glyph cluster.

Binarization – Conversion of a grayscale image to black and white; in this work performed with Otsu thresholding.

Character Error Rate (CER) – Levenshtein edit distance between the predicted and ground-truth text, normalized by the length of the ground truth.

Connectionist Temporal Classification (CTC) – A loss function that allows a sequence model to map unsegmented visual features directly to text without character-level alignment.

Medial Axis Transform (MAT) – A representation that reduces a shape to its one-pixel-wide skeleton, preserving the topology of the original strokes.

Page Segmentation Mode (PSM) – A Tesseract setting that tells the engine what layout to expect, for example PSM 8 (single word) or PSM 13 (raw single line).

Pili – Sinhala dependent vowel modifiers that attach above, below, before or after a base consonant.

Skeletonization (thinning) – Iterative removal of boundary pixels from a binary shape until only a one-pixel-wide skeleton remains; here the Zhang-Suen algorithm.

Zero Width Joiner (ZWJ) – The Unicode control character U+200D, required to form conjunct consonants in Sinhala text.

APPENDICES

This section contains supplementary material that supports the main thesis. Appendix A is cited in Section 5.2, and Appendix B contains the AI use disclosure referred to in the declaration.

Appendix A: Research Instruments and Extended Results

Table A.1 maps each processing stage of Component 1 to the source module that implements it.

Table 6: Mapping of Component 1 layers to source modules

Layer	Module	Responsibility
l3_segment	article_splitter_p1.py	PaddleOCR PPStructure layout detection; article and sub-region (title, text, image) classification; safety buffering around headline boxes
l3_segment	closeup.py, layout.py	Close-up fallback using ink projection profiles and line heights (headline_bands(), headline_for_block())
l3_segment	article_splitter_p2.py	CRAFT TextDetection; tight polygon extraction for text lines
l4a_title	title_extractor.py (stage 1)	Perspective warping of CRAFT crops and horizontal line stitching
l4a_title	title_extractor.py (stage 2)	Rescaling to 384 px height, grayscale conversion, Otsu thresholding, polarity correction, speckle filtering
l4a_title	title_extractor.py (stage 3)	MAT skeletonization (Zhang-Suen) and disk(1) dilation to a 3-pixel stroke
l4a_title	title_extractor.py (stage 4), title.py	Tesseract recognition with sin_raw and sin_custom; confidence-based hybrid routing

Source: Author's implementation (system/layers/).

Perceptual survey protocol. Each trial presented two images, one from a font on which MAT excelled and one from a font on which RAW excelled and asked which looked more complex and harder to recognize. Raters were not told which image was which, left and right placement was randomized for every trial, and a participant log ensured that no rater saw the same pair twice. Twenty raters completed all 20 comparisons and one rater stopped after 4, giving 404 trials (808 image presentations).

Appendix B: Academic Integrity and AI Use Disclosure

Table 7: Disclosure of AI and automated tool use

Tool	Purpose	Verification performed	Sections affected
Gemini	Search globally for approaches/alternatives from its access to google infrastructure.	All proposed methods were individually researched upon, validated and used/rejected.	Section 2 Literature review and related work Section 3 Research methodology
Claude	Search for misalignment/ plot holes in information compilation.	When combining information, the plot holes/misalignments found by Claude were verified manually and corrected in the required circumstances, sometimes going back and re-running the results	Section 6 Results and Validation

Source: Author.